Binary Classification:

1. Suppose we have data:

x_1	x_2	**Class**
1	1	0
1.5	1	0
2	1.5	0
2.5	2	0
7	7	1
8	7	1
8	8	1
9	8	1

We have:

$$X = \begin{bmatrix} 1 & 1 \\ 1.5 & 1 \\ 2 & 1.5 \\ 2.5 & 2 \\ 7 & 7 \\ 8 & 7 \\ 8 & 8 \\ 9 & 8 \end{bmatrix}$$

and

$$y = \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 1 \\ 1 \end{bmatrix}$$

For a linear classifier, the boundary has the form

$$w_1 x_1 + w_2 x_2 + b = 0$$

$$\boxed{w^T x + b = 0}$$

01_dataset.py

```
import numpy as np

# ========================================
# Dataset
# ========================================

X = np.array([
    [1, 1],
    [1.5, 1],
    [2, 1.5],
    [2.5, 2],

    [7, 7],
    [8, 7],
    [8, 8],
    [9, 8]
])

# ========================================
# Labels
# ========================================

y = np.array([
    0,
    0,
    0,
    0,

    1,
    1,
    1,
    1
])
```

```
# ========================================
# Model parameters
# ========================================

w = np.array([1, 1])
b = -5.0

# ========================================
# Linear model
# z = Xw + b
# ========================================

z = X @ w + b

# ========================================
# Prediction
# ========================================

y_pred = (z >= 0).astype(int)

# ========================================
# Display
# ========================================

print("X:")
print(X)

print("\ny:")
print(y)

print("\nX shape:")
print(X.shape)

print("\ny shape:")
print(y.shape)

print("\nWeights:")
print(w)

print("\nBias:")
print(b)

print("\nLinear scores:")
print(z)

print("\nPredictions:")
print(y_pred)
```

For w1 = 1, w2 = 1, b = -5.0

```
PS C:\Users\Lenovo\ML Journey\01_binary_classification> py 01_dataset.py
X:
[[1.  1. ]
 [1.5 1. ]
 [2.  1.5]
 [2.5 2. ]
 [7.  7. ]
 [8.  7. ]
 [8.  8. ]
 [9.  8. ]]

y:
[0 0 0 0 1 1 1 1]

X shape:
(8, 2)

y shape:
(8,)

Weights:
[1 1]

Bias:
-5.0

Linear scores:
[-3.  -2.5 -1.5 -0.5  9.  10.  11.  12. ]

Predictions:
[0 0 0 0 1 1 1 1]
```

No learning yet!

2. **0/1 loss**

$$L = \begin{cases} 0 & \hat{y} = y \\ 1 & \hat{y} \neq y \end{cases}$$

So:

Actual	Prediction	Loss
0	0	0
0	1	1
1	0	1
1	1	0

But problem occurs when, suppose our model produces: $z = 0.01$ and therefore predicts $\hat{y} = 1$
Now suppose another model produces: $z = 10$ it also predicts $\hat{y} = 1$
z = 0.01 looks → barely confident
z = 10 → extremely confident
But both predict class 1.

3. **sigmoid**
Our linear model produces: $z = w^T x + b$
But z can be anything: $-\infty < z < \infty$
Therefor we want: $0 \leq p \leq 1$
So we need a function that converts any real number into a probability.
That's the **sigmoid function**:

$$\boxed{\sigma(z) = \frac{1}{1 + e^{-z}}}$$

Previously:

$$X \rightarrow z \rightarrow \hat{y}$$

Now:

$$\boxed{X \rightarrow z \rightarrow p \rightarrow \hat{y}}$$

where

$$z = Xw + b$$
$$p = \sigma(z)$$

and finally:

$$\hat{y} = \begin{cases} 1 & p \geq 0.5 \\ 0 & p < 0.5 \end{cases}$$

This is the basic structure of **logistic classification**.

02_sigmoid.py

```
import numpy as np

# ========================================
# Dataset
# ========================================

X = np.array([
    [1, 1],
    [1.5, 1],
    [2, 1.5],
    [2.5, 2],

    [7, 7],
    [8, 7],
    [8, 8],
    [9, 8]
])

y = np.array([
    0,
    0,
    0,
    0,

    1,
    1,
    1,
    1
])
```

```
# ========================================
# Parameters
# ========================================

w = np.array([2.0, 2.0])
b = -10.0

# ========================================
# Sigmoid
# ========================================

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# ========================================
# Linear score
# ========================================

z = X @ w + b

# ========================================
# Probability
# ========================================

p = sigmoid(z)

# ========================================
# Prediction
# ========================================

y_pred = (p >= 0.5).astype(int)

# ========================================
# Output
# ========================================

print("Linear scores:")
print(z)

print("\nProbabilities:")
print(p)

print("\nPredictions:")
print(y_pred)

print("\nActual labels:")
print(y)
```

```
PS C:\Users\Lenovo\ML Journey\01_binary_classification> py 02_sigmoid.py
Linear scores:
[-6. -5. -3. -1. 18. 20. 22. 24.]

Probabilities:
[0.00247262 0.00669285 0.04742587 0.26894142 0.99999998 1.
 1.         1.        ]

Predictions:
[0 0 0 0 1 1 1 1]

Actual labels:
[0 0 0 0 1 1 1 1]
PS C:\Users\Lenovo\ML Journey\01_binary_classification>
```

4. Binary Cross-Entropy

$$\boxed{L = -[y \log(p) + (1 - y) \log(1 - p)]}$$

Case 1: Actual class = 1
If: $y = 1$ then: $L = -[1 \log(p) + 0 \log(1 - p)]$
Therefore: $\boxed{L = -\log(p)}$
If $p = 0.9$ then $L = -\log(0.9) \approx 0.105$ Small loss.

If $p = 0.5$ then $L = -\log(0.5) \approx 0.693$ Larger loss.
If $p = 0.1$then $L = -\log(0.1) \approx 2.303$ Large loss.
If $p = 0.01$ then $L = -\log(0.01) \approx 4.605$ Huge loss.

Case 2: Actual class = 0
If $y = 0$ The loss becomes: $L = -\log(1 - p)$
So:
If $p = 0.1$ then $L = -\log(0.9) \approx 0.105$ Small loss.
If $p = 0.5$ then $L = -\log(0.5) \approx 0.693$ Larger loss.
If $p = 0.9$ then $L = -\log(0.1) \approx 2.303$ Large loss.
If $p = 0.99$ then $L = -\log(0.01) \approx 4.605$ Huge loss.

Therefore for both cases: Confidently wrong → heavily punished.

For N samples:

$$J = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log p_i + (1 - y_i)\log(1 - p_i)]$$

$J$is commonly called the cost, objective, or empirical loss.

03 loss.py

```
import numpy as np

# ========================================
# Dataset
# ========================================

X = np.array([
    [1, 1],
    [1.5, 1],
    [2, 1.5],
    [2.5, 2],

    [7, 7],
    [8, 7],
    [8, 8],
    [9, 8]
])

y = np.array([
    0,
    0,
    0,
    0,
    1,
    1,
    1,
    1
])

# ========================================
# Parameters
# ========================================

w = np.array([1.0, 1.0])
b = -5.0

# ========================================
# Sigmoid
# ========================================

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# ========================================
# Binary Cross-Entropy
# ========================================

def binary_cross_entropy(y, p):
```

```
    epsilon = 1e-15

    p = np.clip(p, epsilon, 1 - epsilon)

    loss = -(y * np.log(p) +
             (1 - y) * np.log(1 - p))

    return np.mean(loss)

# ========================================
# Forward pass
# ========================================

z = X @ w + b

p = sigmoid(z)

# ========================================
# Loss
# ========================================

loss = binary_cross_entropy(y, p)

# ========================================
# Output
# ========================================

print("Scores:")
print(z)

print("\nProbabilities:")
print(p)

print("\nActual labels:")
print(y)

print("\nAverage BCE loss:")
print(loss)
```

```
PS C:\Users\Lenovo\ML Journey\01_binary_classification> py 03_loss.py
Scores:
[-3.  -2.5 -1.5 -0.5  9.  10.  11.  12. ]

Probabilities:
[0.04742587 0.07585818 0.18242552 0.37754067 0.99987661 0.9999546
 0.9999833  0.99999386]

Actual labels:
[0 0 0 0 1 1 1 1]

Average BCE loss:
0.10039487435911085
```

5. **Gradient Descent**:

$$\boxed{\theta \leftarrow \theta - \eta \nabla J}$$

where: η is the learning rate.
For weights:

$$w_j \leftarrow w_j - \eta \frac{\partial J}{\partial w_j}$$

and:

$$b \leftarrow b - \eta \frac{\partial J}{\partial b}$$

The learning rate controls the step size.

Too small	Very slow learning
Too large	Overshooting Instability Possibly divergence
Reasonable	Gradual movement toward minimum

For our model we have: $z = w^T x + b$
Then: $p = \sigma(z)$ where: $\sigma(z) = \frac{1}{1+e^{-z}}$
And our loss is: $L = -[y \log(p) + (1-y)\log(1-p)]$
We want: $\frac{\partial L}{\partial w}$ and: $\frac{\partial L}{\partial b}$
Chain rule: $\boxed{\frac{\partial L}{\partial w} = \frac{\partial L}{\partial p}\frac{\partial p}{\partial z}\frac{\partial z}{\partial w}}$

Step 1 — Derivative of loss with respect to probability

$$L = -[y \log p + (1-y)\log(1-p)]$$

Differentiate with respect to p:

$$\frac{\partial L}{\partial p} = -\left[\frac{y}{p} + (1-y)\frac{-1}{1-p}\right]$$

Therefore:

$$\boxed{\frac{\partial L}{\partial p} = -\frac{y}{p} + \frac{1-y}{1-p}}$$

Step 2 — Derivative of sigmoid

$$p = \sigma(z)$$

The derivative of sigmoid is:

$$\boxed{\frac{\partial p}{\partial z} = p(1-p)}$$

Step 3 — Combine them

Using the chain rule:

$$\frac{\partial L}{\partial z} = \frac{\partial L}{\partial p}\frac{\partial p}{\partial z}$$

$$\frac{\partial L}{\partial z} = \left(-\frac{y}{p} + \frac{1-y}{1-p}\right)p(1-p)$$

$$\boxed{\frac{\partial L}{\partial z} = p - y}$$

Step 4 — Derivative of z with respect to w, b

$$z = w^T x + b$$

For two features:

$$z = w_1 x_1 + w_2 x_2 + b$$

Therefore:

$$\frac{\partial z}{\partial w_1} = x_1$$

and:

$$\frac{\partial z}{\partial w_2} = x_2$$

So:

$$\boxed{\frac{\partial z}{\partial w} = x}$$

And for the bias:

$$\frac{\partial z}{\partial b} = 1$$

Therefore, gradient for one sample

$$\frac{\partial L}{\partial w} = \frac{\partial L}{\partial z}\frac{\partial z}{\partial w}$$

Since:

$$\frac{\partial L}{\partial z} = p - y$$

we get:

$$\boxed{\frac{\partial L}{\partial w} = (p-y)x}$$

And for the bias:

$$\frac{\partial z}{\partial b} = 1$$

so:

$$\boxed{\frac{\partial L}{\partial b} = p - y}$$

For our complete dataset:

$$X \in \mathbb{R}^{N \times D}$$

we have:

$$z = Xw + b$$

and:

$$p = \sigma(z)$$

For the average loss:

$$J = \frac{1}{N}\sum_i L_i$$

the weight gradient becomes:

$$\boxed{\nabla_w J = \frac{1}{N} X^T (p - y)}$$

and bias gradient:

$$\boxed{\frac{\partial J}{\partial b} = \frac{1}{N}\sum_i (p_i - y_i)}$$

04_gradient.py

```
import numpy as np

# ========================================
# Dataset
# ========================================

X = np.array([
    [1, 1],
    [1.5, 1],
    [2, 1.5],
    [2.5, 2],

    [7, 7],
    [8, 7],
    [8, 8],
    [9, 8]
])

y = np.array([
    0,
    0,
    0,
    0,
    1,
    1,
    1,
    1
])

# ========================================
# Parameters
# ========================================

w = np.array([1.0, 1.0])
b = -5.0

# ========================================
# Sigmoid
# ========================================

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# ========================================
# Forward pass
# ========================================
```

```
z = X @ w + b

p = sigmoid(z)

# ========================================
# Gradient
# ========================================

error = p - y

dw = (X.T @ error) / len(X)

db = np.mean(error)

# ========================================
# Output
# ========================================

print("Scores:")
print(z)

print("\nProbabilities:")
print(p)

print("\nActual labels:")
print(y)

print("\nError (p - y):")
print(error)

print("\nGradient dw:")
print(dw)

print("\nGradient db:")
print(db)
```

```
PS C:\Users\Lenovo\ML Journey\01_binary_classification> py 04_gradient.py
Scores:
[-3.  -2.5 -1.5 -0.5  9.  10.  11.  12. ]

Probabilities:
[0.04742587 0.07585818 0.18242552 0.37754067 0.99987661 0.9999546
 0.9999833  0.99999386]

Actual labels:
[0 0 0 0 1 1 1 1]

Error (p - y):
[ 4.74258732e-02  7.58581800e-02  1.82425524e-01  3.77540669e-01
 -1.23394576e-04 -4.53978687e-05 -1.67014218e-05 -6.14417460e-06]

Gradient dw:
[0.1835625  0.14382992]

Gradient db:
0.08538232597027165
```

05_gradient_descent.py

```
import numpy as np

# ========================================
# Dataset
# ========================================

X = np.array([
    [1, 1],
    [1.5, 1],
    [2, 1.5],
    [2.5, 2],

    [7, 7],
    [8, 7],
    [8, 8],
    [9, 8]
```

```
])

y = np.array([
    0,
    0,
    0,
    0,
    1,
    1,
    1,
    1
])

# ========================================
# Initial parameters
# ========================================

w = np.array([1.0, 1.0])
b = -5.0

# ========================================
# Learning rate
# ========================================

learning_rate = 0.1

# ========================================
# Sigmoid
# ========================================

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# ========================================
# Loss
# ========================================

def binary_cross_entropy(y, p):

    epsilon = 1e-15

    p = np.clip(p, epsilon, 1 - epsilon)

    loss = -(y * np.log(p) +
             (1 - y) * np.log(1 - p))

    return np.mean(loss)

# ========================================
# Forward pass
# ========================================

z = X @ w + b

p = sigmoid(z)

# ========================================
# Loss before update
# ========================================

loss_before = binary_cross_entropy(y, p)

# ========================================
# Gradient
# ========================================

error = p - y

dw = (X.T @ error) / len(X)

db = np.mean(error)

# ========================================
# Gradient descent update
# ========================================
```

```
w = w - learning_rate * dw

b = b - learning_rate * db

# ========================================
# Forward pass again
# ========================================

z_new = X @ w + b

p_new = sigmoid(z_new)

# ========================================
# Loss after update
# ========================================

loss_after = binary_cross_entropy(y, p_new)

# ========================================
# Results
# ========================================

print("Gradient dw:")
print(dw)

print("\nGradient db:")
print(db)

print("\nUpdated weights:")
print(w)

print("\nUpdated bias:")
print(b)

print("\nLoss before update:")
print(loss_before)

print("\nLoss after update:")
print(loss_after)
```

For learning rate 0.1 and one GD update:

```
PS C:\Users\Lenovo\ML Journey\01_binary_classification> py 05_gradient_descent.py
Gradient dw:
[0.1835625  0.14382992]

Gradient db:
0.08538232597027165

Updated weights:
[0.98164375 0.98561701]

Updated bias:
-5.008538232597028

Loss before update:
0.10039487435911085

Loss after update:
0.09438616376568058
```

06 training.py

```
import numpy as np

# ========================================
# Dataset
# ========================================

X = np.array([
    [1, 1],
    [1.5, 1],
    [2, 1.5],
    [2.5, 2],
```

```
    [7, 7],
    [8, 7],
    [8, 8],
    [9, 8]
])

y = np.array([
    0,
    0,
    0,
    0,
    1,
    1,
    1,
    1
])

# ========================================
# Parameters
# ========================================

w = np.array([0.0, 0.0])
b = 0.0

learning_rate = 0.1
epochs = 1000

# ========================================
# Functions
# ========================================

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def binary_cross_entropy(y, p):

    epsilon = 1e-15

    p = np.clip(p, epsilon, 1 - epsilon)

    loss = -(y * np.log(p) +
             (1 - y) * np.log(1 - p))

    return np.mean(loss)

# ========================================
# Training
# ========================================

for epoch in range(epochs):

    # Forward pass
    z = X @ w + b

    p = sigmoid(z)

    # Calculate loss
    loss = binary_cross_entropy(y, p)

    # Calculate gradient
    error = p - y

    dw = (X.T @ error) / len(X)

    db = np.mean(error)

    # Update parameters
    w = w - learning_rate * dw

    b = b - learning_rate * db

    # Print progress
    if epoch % 100 == 0:
        print(
            f"Epoch {epoch}: "
```

```
            f"Loss = {loss:.6f}"
        )

# ========================================
# Final prediction
# ========================================

z = X @ w + b

p = sigmoid(z)

y_pred = (p >= 0.5).astype(int)

# ========================================
# Results
# ========================================

print("\nFinal weights:")
print(w)

print("\nFinal bias:")
print(b)

print("\nFinal probabilities:")
print(p)

print("\nPredictions:")
print(y_pred)

print("\nActual:")
print(y)
```

```
PS C:\Users\Lenovo\ML Journey\01_binary_classification> py 06_training.py
Epoch 0: Loss = 0.693147
Epoch 100: Loss = 0.214431
Epoch 200: Loss = 0.124401
Epoch 300: Loss = 0.086077
Epoch 400: Loss = 0.065440
Epoch 500: Loss = 0.052672
Epoch 600: Loss = 0.044034
Epoch 700: Loss = 0.037815
Epoch 800: Loss = 0.033130
Epoch 900: Loss = 0.029477

Final weights:
[-0.25490388  1.57082682]

Final bias:
-4.984244746093294

Final probabilities:
[0.02488423 0.02197191 0.0415742  0.07728284 0.98561693 0.98151816
 0.99610104 0.99497471]

Predictions:
[0 0 0 0 1 1 1 1]

Actual:
[0 0 0 0 1 1 1 1]
```

-

07_visualize_learning.py

```
import numpy as np
import matplotlib.pyplot as plt

# ============================================================
# 1. DATASET
# ============================================================

X = np.array([
    [1, 1],
    [1.5, 1],
```

```
    [2, 1.5],
    [2.5, 2],
    [7, 7],
    [8, 7],
    [8, 8],
    [9, 8]
])

y = np.array([0, 0, 0, 0, 1, 1, 1, 1])

# ============================================================
# 2. FUNCTIONS
# ============================================================

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def binary_cross_entropy(y, p):

    epsilon = 1e-15

    p = np.clip(p, epsilon, 1 - epsilon)

    loss = -(y * np.log(p) +
             (1 - y) * np.log(1 - p))

    return np.mean(loss)

# ============================================================
# 3. TRAINING
# ============================================================

w = np.array([0.0, 0.0])
b = 0.0

learning_rate = 0.1
epochs = 1000

loss_history = []

for epoch in range(epochs):

    # Forward pass
    z = X @ w + b

    p = sigmoid(z)

    # Calculate loss
    loss = binary_cross_entropy(y, p)

    loss_history.append(loss)

    # Gradient
    error = p - y

    dw = (X.T @ error) / len(X)

    db = np.mean(error)

    # Update parameters
    w = w - learning_rate * dw

    b = b - learning_rate * db

# ============================================================
# 4. FINAL PREDICTIONS
# ============================================================

z = X @ w + b

p = sigmoid(z)

y_pred = (p >= 0.5).astype(int)

print("Final weights:", w)
print("Final bias:", b)
```

```
print("\nProbabilities:")
print(p)

print("\nPredictions:")
print(y_pred)

print("\nActual:")
print(y)

# ============================================================
# 5. PLOT 1 — LOSS CURVE
# ============================================================

plt.figure(figsize=(8, 5))

plt.plot(loss_history)

plt.xlabel("Epoch")
plt.ylabel("Binary Cross-Entropy Loss")

plt.title("Learning Process: Loss vs Epoch")

plt.grid(True)

plt.tight_layout()

# ============================================================
# 6. PLOT 2 — DECISION BOUNDARY
# ============================================================

plt.figure(figsize=(8, 6))

# Class 0
plt.scatter(
    X[y == 0, 0],
    X[y == 0, 1],
    label="Class 0"
)

# Class 1
plt.scatter(
    X[y == 1, 0],
    X[y == 1, 1],
    label="Class 1"
)

# Generate x1 values
x1_values = np.linspace(0, 10, 100)

# Decision boundary:
#
# w1*x1 + w2*x2 + b = 0
#
# Therefore:
#
# x2 = -(w1*x1 + b) / w2

x2_values = -(w[0] * x1_values + b) / w[1]

plt.plot(
    x1_values,
    x2_values,
    label="Decision Boundary"
)

plt.xlabel("Feature 1")
plt.ylabel("Feature 2")

plt.title("Learned Decision Boundary")

plt.legend()

plt.grid(True)

plt.tight_layout()
```

```
# ============================================================
# 7. SHOW BOTH FIGURES
# ============================================================

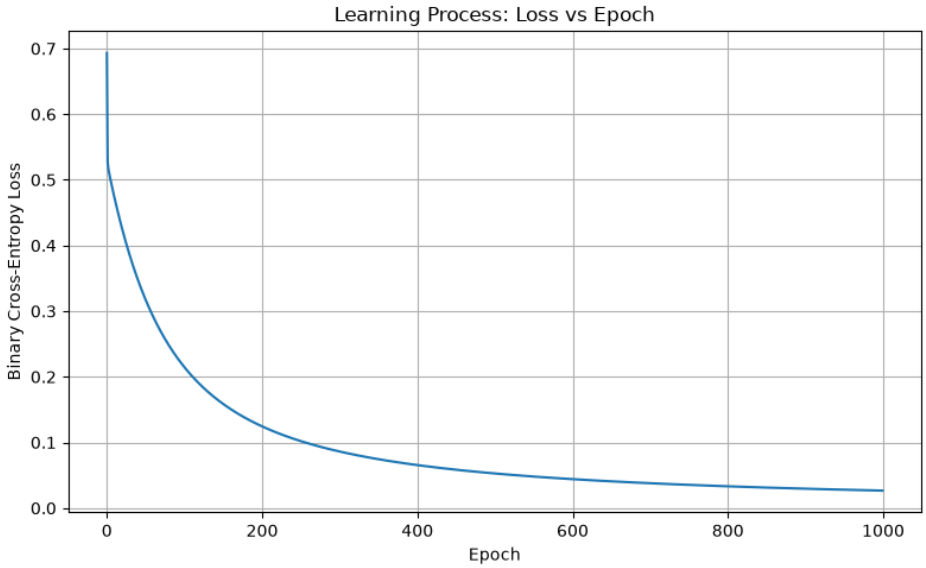
plt.show()
```

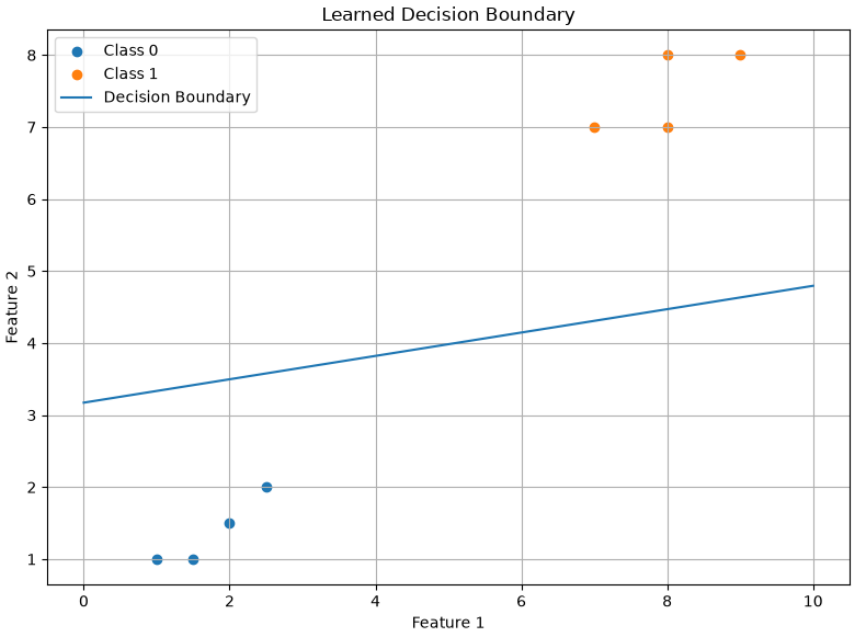
```
PS C:\Users\Lenovo\ML Journey\01_binary_classification> py 07_visualize_learning.py
Final weights: [-0.25490388  1.57082682]
Final bias: -4.984244746093294

Probabilities:
[0.02488423 0.02197191 0.0415742  0.07728284 0.98561693 0.98151816
 0.99610104 0.99497471]

Predictions:
[0 0 0 0 1 1 1 1]

Actual:
[0 0 0 0 1 1 1 1]
```





08_learning_boundary.py

```
import numpy as np
import matplotlib.pyplot as plt

# ============================================================
# 1. DATASET
# ============================================================

X = np.array([
    [1, 1],
    [1.5, 1],
    [2, 1.5],
    [2.5, 2],
    [7, 7],
    [8, 7],
    [8, 8],
    [9, 8]
])

y = np.array([0, 0, 0, 0, 1, 1, 1, 1])

# ============================================================
# 2. FUNCTIONS
# ============================================================

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def binary_cross_entropy(y, p):

    epsilon = 1e-15

    p = np.clip(p, epsilon, 1 - epsilon)

    loss = -(y * np.log(p) +
             (1 - y) * np.log(1 - p))

    return np.mean(loss)

# ============================================================
# 3. TRAINING SETTINGS
# ============================================================

learning_rate = 0.1
epochs = 1000

w = np.array([0.0, 0.0])
b = 0.0

# Epochs that we want to observe
snapshot_epochs = [0, 10, 50, 100, 500, 1000]

snapshots = {}

# ============================================================
# 4. TRAINING
# ============================================================

for epoch in range(epochs + 1):

    # ----------------------------------------
    # Forward pass
    # ----------------------------------------

    z = X @ w + b

    p = sigmoid(z)

    loss = binary_cross_entropy(y, p)

    # ----------------------------------------
    # Save model state
    # ----------------------------------------

    if epoch in snapshot_epochs:
```

```
        snapshots[epoch] = {
            "w": w.copy(),
            "b": b,
            "loss": loss
        }

    # ---------------------------------------
    # Gradient
    # ---------------------------------------

    error = p - y

    dw = (X.T @ error) / len(X)

    db = np.mean(error)

    # ---------------------------------------
    # Gradient descent
    # ---------------------------------------

    w = w - learning_rate * dw

    b = b - learning_rate * db

# ============================================================
# 5. PRINT SNAPSHOT INFORMATION
# ============================================================

print("\nLEARNING PROCESS")
print("=" * 50)

for epoch in snapshot_epochs:

    w_snapshot = snapshots[epoch]["w"]
    b_snapshot = snapshots[epoch]["b"]
    loss_snapshot = snapshots[epoch]["loss"]

    print(
        f"Epoch {epoch:4d} | "
        f"w = {w_snapshot} | "
        f"b = {b_snapshot:.4f} | "
        f"Loss = {loss_snapshot:.6f}"
    )

# ============================================================
# 6. CREATE VISUALIZATION
# ============================================================

fig, axes = plt.subplots(
    2,
    3,
    figsize=(14, 8)
)

# Generate x-axis values
x1_values = np.linspace(0, 10, 100)

for ax, epoch in zip(axes.ravel(), snapshot_epochs):

    # ---------------------------------------------------------
    # Get saved model
    # ---------------------------------------------------------

    w_snapshot = snapshots[epoch]["w"]
    b_snapshot = snapshots[epoch]["b"]

    loss_snapshot = snapshots[epoch]["loss"]

    # ---------------------------------------------------------
    # Plot Class 0
    # ---------------------------------------------------------

    ax.scatter(
        X[y == 0, 0],
        X[y == 0, 1],
        label="Class 0"
```

```
    )

    # ----------------------------------------------------------
    # Plot Class 1
    # ----------------------------------------------------------

    ax.scatter(
        X[y == 1, 0],
        X[y == 1, 1],
        label="Class 1"
    )

    # ----------------------------------------------------------
    # Decision boundary
    #
    # w1*x1 + w2*x2 + b = 0
    #
    # x2 = -(w1*x1 + b)/w2
    # ----------------------------------------------------------

    if abs(w_snapshot[1]) > 1e-10:

        x2_values = (
            -(w_snapshot[0] * x1_values + b_snapshot)
            / w_snapshot[1]
        )

        ax.plot(
            x1_values,
            x2_values,
            label="Decision Boundary"
        )

    # ----------------------------------------------------------
    # Formatting
    # ----------------------------------------------------------

    ax.set_xlim(0, 10)
    ax.set_ylim(0, 10)

    ax.set_xlabel("Feature 1")
    ax.set_ylabel("Feature 2")

    ax.set_title(
        f"Epoch {epoch}\nLoss = {loss_snapshot:.4f}"
    )

    ax.grid(True)

    ax.legend()

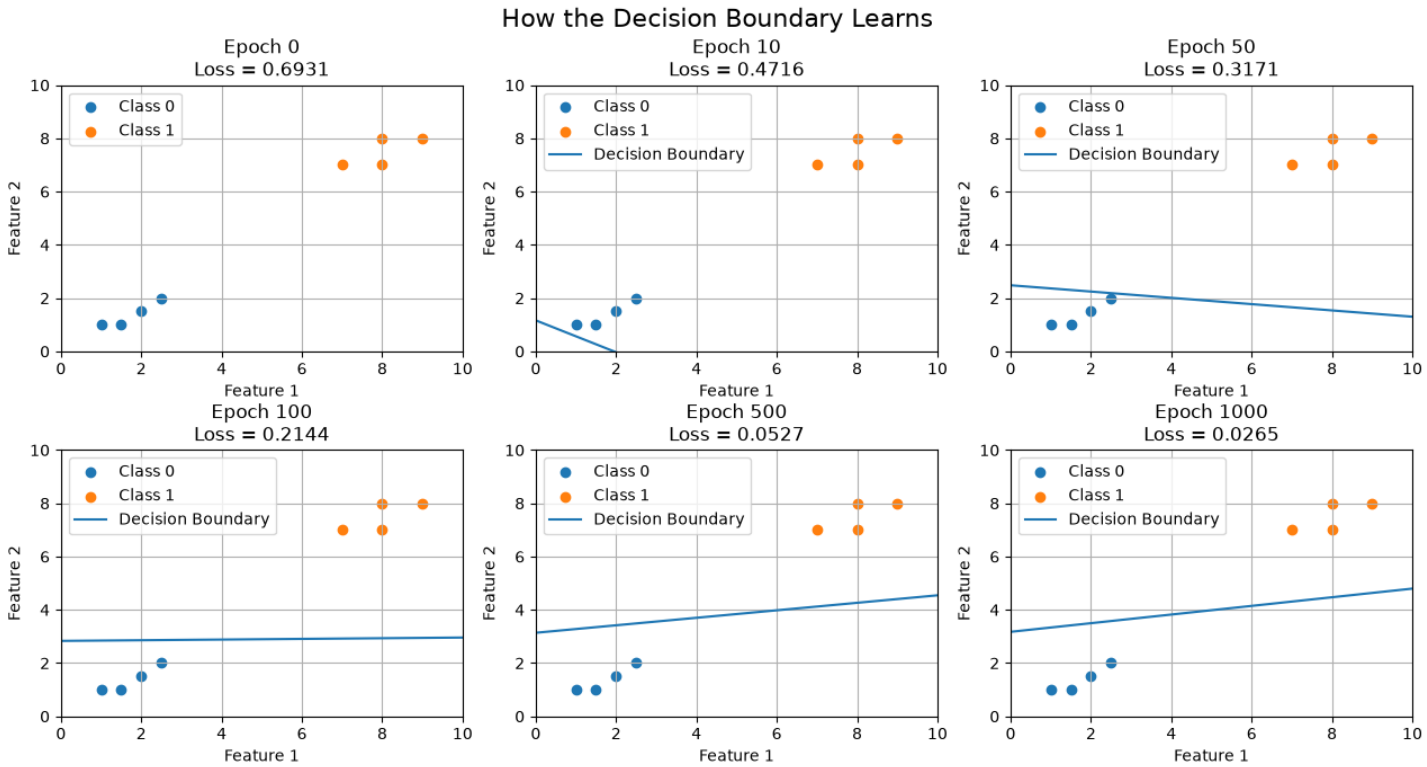
plt.suptitle(
    "How the Decision Boundary Learns",
    fontsize=16
)

plt.tight_layout()

plt.show()
```

```
PS C:\Users\Lenovo\ML Journey\01_binary_classification> py 08_learning_boundary.py

LEARNING PROCESS
==================================================
Epoch    0 | w = [0. 0.] | b = 0.0000 | Loss = 0.693147
Epoch   10 | w = [0.1073807 0.1794488] | b = -0.2089 | Loss = 0.471597
Epoch   50 | w = [0.04578776 0.38793108] | b = -0.9629 | Loss = 0.317072
Epoch  100 | w = [-0.00733332  0.58196388] | b = -1.6483 | Loss = 0.214431
Epoch  500 | w = [-0.17545991  1.24735712] | b = -3.9135 | Loss = 0.052672
Epoch 1000 | w = [-0.25490388  1.57082682] | b = -4.9842 | Loss = 0.026550
```



6. **Evaluation of the Classifier**

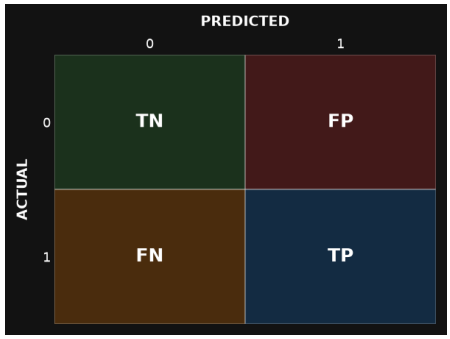
The four possible outcomes

Actual class 1, Predicted 1: $\boxed{TP = \text{True Positive}}$

Actual class 0, predicted 0: $\boxed{TN = \text{True Negative}}$

Actual class 0, predicted 1: $\boxed{FP = \text{False Positive}}$

Actual class 1, predicted 0: $\boxed{FN = \text{False Negative}}$



This is the confusion matrix.

Accuracy

Accuracy asks: Out of all predictions, how many were correct?

$$\boxed{Accuracy = \frac{TP + TN}{TP + TN + FP + FN}}$$

Precision

Precision asks: When the model predicts positive, how often is it correct?

$$\boxed{Precision = \frac{TP}{TP + FP}}$$

High precision means few false positives.

Recall

Recall asks: Of all the actual positive samples, how many did we find?

$$\boxed{Recall = \frac{TP}{TP + FN}}$$

High recall means few false negatives.

F1 Score
Sometimes you want a balance between precision and recall. The harmonic mean is used:

$$\boxed{F1 = 2\frac{Precision \cdot Recall}{Precision + Recall}}$$

Equivalent form:

$$\boxed{F1 = \frac{2TP}{2TP + FP + FN}}$$

Recall is also called **True Positive Rate**:

$$\boxed{TPR = \frac{TP}{TP + FN}}$$

False Positive Rate:

$$\boxed{FPR = \frac{FP}{FP + TN}}$$

09_evaluation.py

```
import numpy as np

# ============================================================
# 1. DATA
# ============================================================

y_true = np.array([
    0, 0, 0, 0,
    1, 1, 1, 1
])

y_pred = np.array([
    0, 0, 1, 0,
    1, 1, 0, 1
])

# ============================================================
# 2. CONFUSION MATRIX COMPONENTS
# ============================================================

TP = np.sum((y_true == 1) & (y_pred == 1))

TN = np.sum((y_true == 0) & (y_pred == 0))

FP = np.sum((y_true == 0) & (y_pred == 1))

FN = np.sum((y_true == 1) & (y_pred == 0))

print("Confusion Matrix Components")
print("===========================")

print("TP =", TP)
print("TN =", TN)
print("FP =", FP)
print("FN =", FN)

# ============================================================
# 3. CONFUSION MATRIX
# ============================================================

confusion_matrix = np.array([
    [TN, FP],
    [FN, TP]
])

print("\nConfusion Matrix:")
print(confusion_matrix)

# ============================================================
# 4. ACCURACY
# ============================================================
```

```
accuracy = (TP + TN) / (TP + TN + FP + FN)

print("\nAccuracy =", accuracy)

# ============================================================
# 5. PRECISION
# ============================================================

precision = TP / (TP + FP)

print("Precision =", precision)

# ============================================================
# 6. RECALL
# ============================================================

recall = TP / (TP + FN)

print("Recall =", recall)

# ============================================================
# 7. F1 SCORE
# ============================================================

f1 = 2 * (precision * recall) / (precision + recall)

print("F1 Score =", f1)
```

```
PS C:\Users\Lenovo\ML Journey\01_binary_classification> py 08_learning_boundary.py

LEARNING PROCESS
================================================
Epoch    0 | w = [0. 0.] | b = 0.0000 | Loss = 0.693147
Epoch   10 | w = [0.1073807 0.1794488] | b = -0.2089 | Loss = 0.471597
Epoch   50 | w = [0.04578776 0.38793108] | b = -0.9629 | Loss = 0.317072
Epoch  100 | w = [-0.00733332  0.58196388] | b = -1.6483 | Loss = 0.214431
Epoch  500 | w = [-0.17545991  1.24735712] | b = -3.9135 | Loss = 0.052672
Epoch 1000 | w = [-0.25490388  1.57082682] | b = -4.9842 | Loss = 0.026550
PS C:\Users\Lenovo\ML Journey\01_binary_classification> py 09_evaluation.py
Confusion Matrix Components
===========================
TP = 3
TN = 3
FP = 1
FN = 1

Confusion Matrix:
[[3 1]
 [1 3]]

Accuracy = 0.75
Precision = 0.75
Recall = 0.75
F1 Score = 0.75
```

10_model_evaluation.py

```
import numpy as np

# ============================================================
# 1. DATASET
# ============================================================

X = np.array([
    [1, 1],
    [1.5, 1],
    [2, 1.5],
    [2.5, 2],
    [7, 7],
    [8, 7],
    [8, 8],
    [9, 8]
])

y = np.array([
    0, 0, 0, 0,
    1, 1, 1, 1
```

```
])

# ============================================================
# 2. FUNCTIONS
# ============================================================

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def binary_cross_entropy(y, p):

    epsilon = 1e-15

    p = np.clip(p, epsilon, 1 - epsilon)

    loss = -(y * np.log(p) +
             (1 - y) * np.log(1 - p))

    return np.mean(loss)

# ============================================================
# 3. TRAIN MODEL
# ============================================================

w = np.array([0.0, 0.0])

b = 0.0

learning_rate = 0.1

epochs = 1000

for epoch in range(epochs):

    # Forward pass
    z = X @ w + b

    p = sigmoid(z)

    # Gradient
    error = p - y

    dw = (X.T @ error) / len(X)

    db = np.mean(error)

    # Update
    w = w - learning_rate * dw

    b = b - learning_rate * db

# ============================================================
# 4. PREDICTION
# ============================================================

z = X @ w + b

probabilities = sigmoid(z)

y_pred = (probabilities >= 0.5).astype(int)

# ============================================================
# 5. CONFUSION MATRIX
# ============================================================

TP = np.sum((y == 1) & (y_pred == 1))

TN = np.sum((y == 0) & (y_pred == 0))

FP = np.sum((y == 0) & (y_pred == 1))

FN = np.sum((y == 1) & (y_pred == 0))

# ============================================================
# 6. METRICS
# ============================================================
```

```python
accuracy = (TP + TN) / len(y)

precision = TP / (TP + FP)

recall = TP / (TP + FN)

f1 = 2 * precision * recall / (precision + recall)

# ============================================================
# 7. RESULTS
# ============================================================

print("Final Model")
print("===========")

print("Weights:", w)

print("Bias:", b)

print("\nProbabilities:")
print(probabilities)

print("\nPredictions:")
print(y_pred)

print("\nActual:")
print(y)

print("\nConfusion Matrix")
print("================")
print(
    np.array([
        [TN, FP],
        [FN, TP]
    ])
)

print("\nMetrics")
print("=======")

print("Accuracy :", accuracy)

print("Precision:", precision)

print("Recall   :", recall)

print("F1 Score :", f1)
```

```
PS C:\Users\Lenovo\ML Journey\01_binary_classification> py 10_model_evaluation.py
Final Model
===========
Weights: [-0.25490388  1.57082682]
Bias: -4.984244746093294

Probabilities:
[0.02488423 0.02197191 0.0415742  0.07728284 0.98561693 0.98151816
 0.99610104 0.99497471]

Predictions:
[0 0 0 0 1 1 1 1]

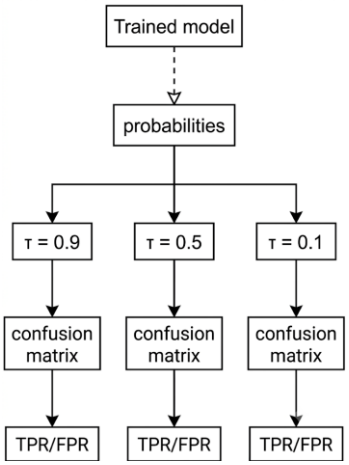
Actual:
[0 0 0 0 1 1 1 1]

Confusion Matrix
================
[[4 0]
 [0 4]]

Metrics
=======
Accuracy : 1.0
Precision: 1.0
Recall   : 1.0
F1 Score : 1.0
```

7. **ROC Curve and AUC**
We previously used: $threshold = \tau = 0.5$
0.5 is just one possible choice.
We could use: $\tau = 0.9$ or $\tau = 0.7$ or $\tau = 0.3$ etc.
Every threshold produces a different confusion matrix.

Example:
Actual labels: 0 0 0 0 1 1 1 1
Predicted probabilities: 0.10 0.20 0.40 0.60 0.65 0.80 0.90 0.95
For threshold = 0.8 Predictions: 0 0 0 0 0 1 1 1
Some positive samples are missed.
Therefore: $FN \uparrow$ and $TPR \downarrow$
For threshold = 0.4 Predictions: 0 0 1 1 1 1 1 1
Now we're detecting more positives.
Therefore: $TPR \uparrow$ But we may also incorrectly classify negatives as positives: $FP \uparrow$ Therefore: $FPR \uparrow$



By ploting all those points: (FPR, TPR) and connecting them, we get the ROC curve.

11_roc_auc.py

```
import numpy as np
import matplotlib.pyplot as plt

# ============================================================
# 1. ACTUAL LABELS
# ============================================================

y_true = np.array([
    0, 0, 0, 0,
    1, 1, 1, 1
])

# ============================================================
# 2. MODEL PROBABILITIES
# ============================================================

y_prob = np.array([
    0.05,
    0.15,
    0.30,
    0.40,
    0.60,
    0.75,
    0.90,
    0.95
])

# ============================================================
# 3. THRESHOLDS
# ============================================================

thresholds = np.linspace(1, 0, 101)
```

```
tpr_values = []
fpr_values = []

# ============================================================
# 4. CALCULATE TPR AND FPR
#    FOR EVERY THRESHOLD
# ============================================================

for threshold in thresholds:

    # Convert probabilities into predictions
    y_pred = (y_prob >= threshold).astype(int)

    # ------------------------------------------------------
    # Confusion matrix
    # ------------------------------------------------------

    TP = np.sum(
        (y_true == 1) &
        (y_pred == 1)
    )

    TN = np.sum(
        (y_true == 0) &
        (y_pred == 0)
    )

    FP = np.sum(
        (y_true == 0) &
        (y_pred == 1)
    )

    FN = np.sum(
        (y_true == 1) &
        (y_pred == 0)
    )

    # ------------------------------------------------------
    # TPR
    # ------------------------------------------------------

    if TP + FN > 0:
        TPR = TP / (TP + FN)
    else:
        TPR = 0

    # ------------------------------------------------------
    # FPR
    # ------------------------------------------------------

    if FP + TN > 0:
        FPR = FP / (FP + TN)
    else:
        FPR = 0

    tpr_values.append(TPR)

    fpr_values.append(FPR)

# Convert lists to NumPy arrays

tpr_values = np.array(tpr_values)

fpr_values = np.array(fpr_values)

# ============================================================
# 5. CALCULATE AUC
# ============================================================

auc = np.trapezoid(
    tpr_values,
    fpr_values
)

print("AUC =", auc)
```

```
# ============================================================
# 6. PLOT ROC CURVE
# ============================================================

plt.figure(figsize=(8, 6))

plt.plot(
    fpr_values,
    tpr_values,
    label=f"ROC Curve (AUC = {auc:.3f})"
)

# Random classifier
plt.plot(
    [0, 1],
    [0, 1],
    linestyle="--",
    label="Random Classifier"
)

plt.xlabel("False Positive Rate (FPR)")

plt.ylabel("True Positive Rate (TPR)")

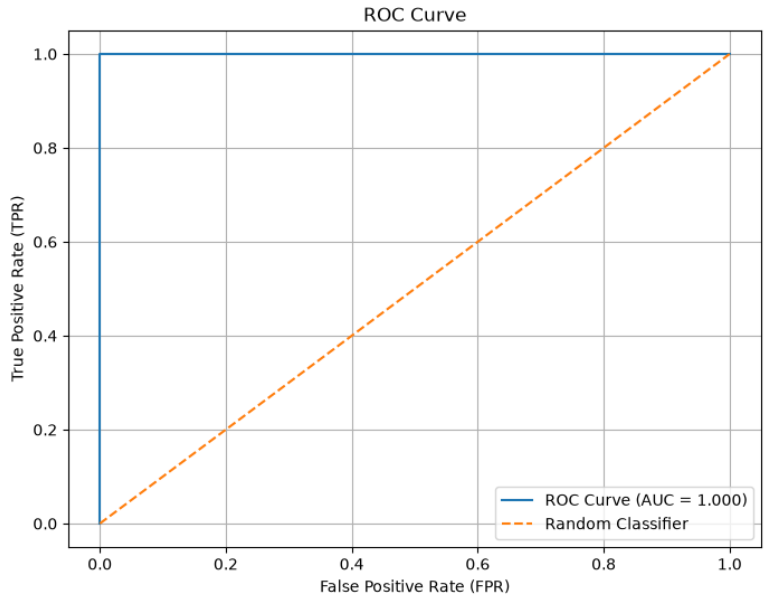
plt.title("ROC Curve")

plt.legend()

plt.grid(True)

plt.show()
```

```
PS C:\Users\Lenovo\ML Journey\01_binary_classification> py 11_roc_auc.py
AUC = 1.0
```



At threshold = 1.0
Almost nothing is predicted positive. Therefore: $TP = 0$ and $FP = 0$
So: $TPR = 0$ and $FPR = 0$
ROC starts near: $(0,0)$

As threshold decreases
More samples become classified as positive. Therefore: TPR generally increases. But: FPR can also increase.

Eventually at **threshold = 0**: Everything becomes positive. Therefore: $TPR = 1$ and $FPR = 1$
ROC ends at: $(1,1)$

AUC means: Area Under the ROC Curve
Using the **trapezoidal rule**. For two neighboring points: (x_1, y_1) and (x_2, y_2)

the approximate area is: $A_i = \frac{y_1+y_2}{2}(x_2 - x_1)$
Then: $AUC = \sum_i A_i$
So AUC is basically numerical integration.

Generally (but not universally):

AUC	Interpretation
1.00	Perfect classifier
0.90–1.00	Excellent
0.80–0.90	Good
0.70–0.80	Reasonable
0.50	Random
< 0.50	Worse than random / ranking reversed

AUC can also be understood as: The probability that the model ranks a randomly chosen positive sample above a randomly chosen negative sample.
For example: $AUC = 0.90$ roughly means that the model has a 90% chance of ranking a random positive higher than a random negative.

8.**The XOR Experiment**
So far, our model has learned: $z = w_1x_1 + w_2x_2 + b$
and therefore, its decision boundary is always: $w_1x_1 + w_2x_2 + b = 0$ which is a **straight line**.
Now we're going to deliberately give it a problem that **no straight line can solve**.

12_xor_failure.py

```
import numpy as np
import matplotlib.pyplot as plt

# ============================================================
# 1. XOR DATASET
# ============================================================

X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
])

y = np.array([
    0,
    1,
    1,
    0
])

# ============================================================
# 2. FUNCTIONS
# ============================================================

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def binary_cross_entropy(y, p):

    epsilon = 1e-15

    p = np.clip(p, epsilon, 1 - epsilon)

    loss = -(y * np.log(p) +
             (1 - y) * np.log(1 - p))

    return np.mean(loss)

# ============================================================
# 3. TRAINING
# ============================================================

w = np.array([0.1, -0.1])
```

```
b = 0.0

learning_rate = 0.1

epochs = 5000

loss_history = []

for epoch in range(epochs):

    # Forward pass
    z = X @ w + b

    p = sigmoid(z)

    # Loss
    loss = binary_cross_entropy(y, p)

    loss_history.append(loss)

    # Gradient
    error = p - y

    dw = (X.T @ error) / len(X)

    db = np.mean(error)

    # Update
    w = w - learning_rate * dw

    b = b - learning_rate * db

# ============================================================
# 4. FINAL PREDICTIONS
# ============================================================

z = X @ w + b

p = sigmoid(z)

y_pred = (p >= 0.5).astype(int)

print("Final weights:", w)

print("Final bias:", b)

print("\nProbabilities:")
print(p)

print("\nPredictions:")
print(y_pred)

print("\nActual:")
print(y)

# ============================================================
# 5. ACCURACY
# ============================================================

accuracy = np.mean(y_pred == y)

print("\nAccuracy:", accuracy)

# ============================================================
# 6. LOSS CURVE
# ============================================================

plt.figure(figsize=(8, 5))

plt.plot(loss_history)

plt.xlabel("Epoch")

plt.ylabel("Loss")

plt.title("XOR: Training Loss")
```

```
plt.grid(True)

plt.show()

# ============================================================
# 7. DECISION BOUNDARY
# ============================================================

plt.figure(figsize=(7, 7))

plt.scatter(
    X[y == 0, 0],
    X[y == 0, 1],
    label="Class 0",
    s=100
)

plt.scatter(
    X[y == 1, 0],
    X[y == 1, 1],
    label="Class 1",
    s=100
)

# Boundary:
#
# w1*x1 + w2*x2 + b = 0
#
# x2 = -(w1*x1+b)/w2

x1_values = np.linspace(-0.2, 1.2, 100)

if abs(w[1]) > 1e-10:

    x2_values = (
        -(w[0] * x1_values + b)
        / w[1]
    )

    plt.plot(
        x1_values,
        x2_values,
        label="Decision Boundary"
    )

plt.xlim(-0.2, 1.2)

plt.ylim(-0.2, 1.2)

plt.xlabel("Feature 1")

plt.ylabel("Feature 2")

plt.title("XOR: Linear Classifier Failure")

plt.legend()

plt.grid(True)

plt.show()

print("\nFinal weights:", w)
print("Final bias:", b)

print("\nXOR Predictions")
print("================")

for i in range(len(X)):
    print(
        f"Input: {X[i]} | "
        f"Probability: {p[i]:.4f} | "
        f"Prediction: {y_pred[i]} | "
        f"Actual: {y[i]}"
    )
```

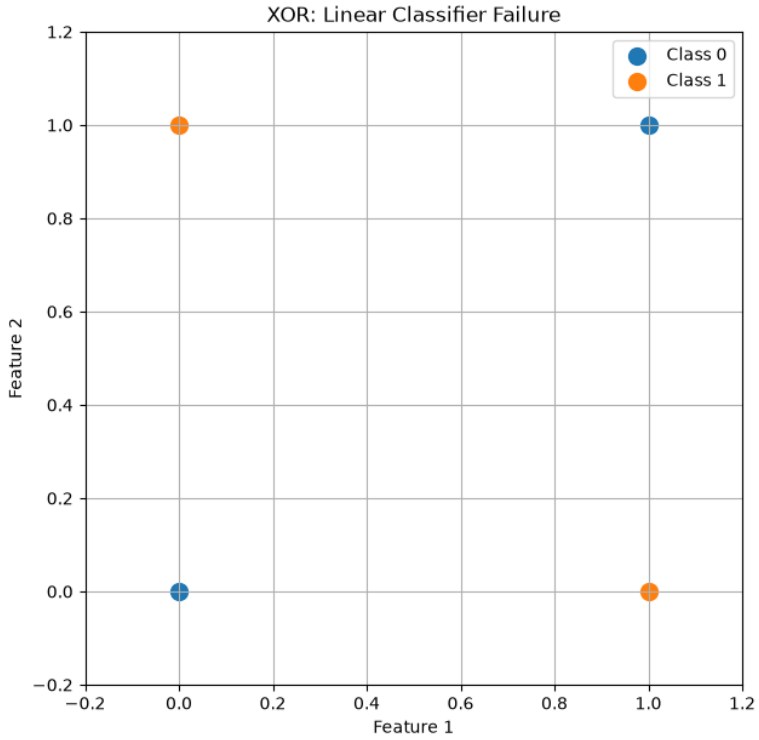
```
PS C:\Users\Lenovo\ML Journey\01_binary_classification> py 12_xor_failure.py
Final weights: [ 2.39629457e-15 -2.42270934e-15]
Final bias: -2.2204460492503132e-17

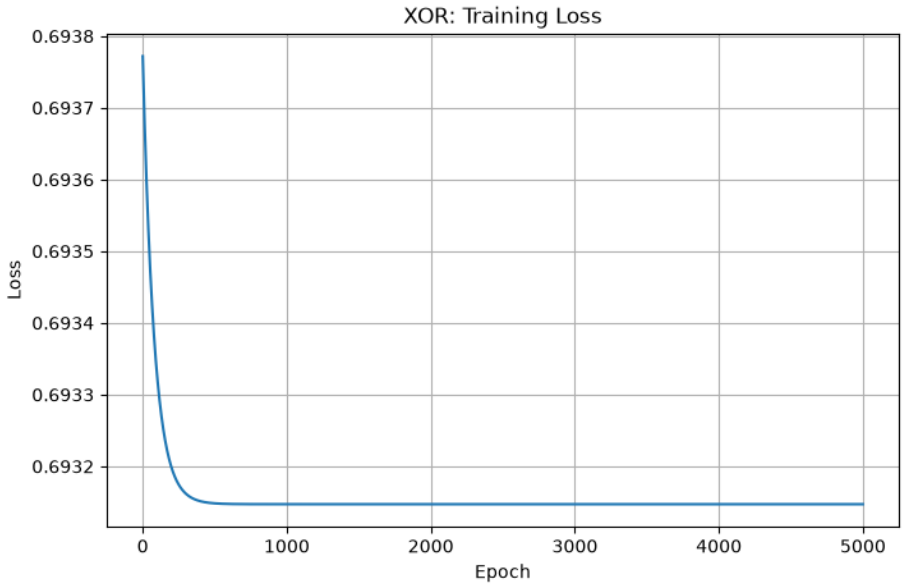
Probabilities:
[0.5 0.5 0.5 0.5]

Predictions:
[1 0 1 1]

Actual:
[0 1 1 0]

Accuracy: 0.25
```





The loss has settled at about: $\boxed{L \approx 0.69315}$
So the logistic regression model has reached the best region it can find for XOR, but cannot solve it.

9. **Building neural network for XOR problem:**
A single neuron
A neuron takes inputs: $x_1, x_2, \dots, x_n$ and weights: $w_1, w_2, \dots, w_n$
Then calculates: $z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$
In vector form: $\boxed{z = w^T x + b}$
Then applies an activation function: $\boxed{a = f(z)}$ (here sigmoid)

13_single_neuron.py

```
import numpy as np

# ============================================================
# INPUT
# ============================================================

x = np.array([2.0, 3.0])

# ============================================================
# WEIGHTS
# ============================================================

w = np.array([0.5, 0.8])

# ============================================================
# BIAS
# ============================================================

b = 0.2

# ============================================================
# LINEAR COMPUTATION
# ============================================================

z = np.dot(x, w) + b

# ============================================================
# SIGMOID ACTIVATION
# ============================================================

a = 1 / (1 + np.exp(-z))

# ============================================================
# RESULTS
# ============================================================

print("Input x:", x)

print("Weights w:", w)

print("Bias b:", b)

print("Weighted sum z:", z)

print("Neuron output a:", a)
```

```
PS C:\Users\Lenovo\ML Journey\01_binary_classification> py 13_single_neuron.py
Input x: [2. 3.]
Weights w: [0.5 0.8]
Bias b: 0.2
Weighted sum z: 3.6000000000000005
Neuron output a: 0.973403006423134
```

Our values are:

$$x = [2,3]$$
$$w = [0.5,0.8]$$
$$b = 0.2$$

Therefore:

$$z = (2)(0.5) + (3)(0.8) + 0.2$$
$$z = 1 + 2.4 + 0.2$$
$$\boxed{z = 3.6}$$

Then sigmoid:

$$a = \frac{1}{1 + e^{-3.6}}$$

approximately:

$$\boxed{a \approx 0.973}$$

So our neuron produces approximately:

Multiple neurons

Suppose we have three hidden neurons.
Instead of calculating:

$$z = w^T x + b$$

once, we calculate it three times.
For example:

$$z_1 = w_{11}x_1 + w_{21}x_2 + b_1$$
$$z_2 = w_{12}x_1 + w_{22}x_2 + b_2$$
$$z_3 = w_{13}x_1 + w_{23}x_2 + b_3$$

Then:

$$a_1 = f(z_1)$$
$$a_2 = f(z_2)$$
$$a_3 = f(z_3)$$

These three activations form a **hidden layer**:

$$\boxed{a = \begin{bmatrix} a_1 \\ a_2 \\ a_3 \end{bmatrix}}$$

Instead of writing three equations individually:

$$z_1 = w_{11}x_1 + w_{21}x_2 + b_1$$

etc., we write:

$$\boxed{z = Wx + b}$$

For three neurons and two inputs:

$$W = \begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \\ w_{31} & w_{32} \end{bmatrix}$$

and:

$$x = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

giving:

$$z = \begin{bmatrix} z_1 \\ z_2 \\ z_3 \end{bmatrix}$$

This is the fundamental operation we'll use to build the network.

14_hidden_layer.py

```
import numpy as np

# ============================================================
# INPUT
# ============================================================

x = np.array([
    [2.0],
    [3.0]
])

# ============================================================
# WEIGHTS
#
# 3 neurons
# 2 inputs per neuron
# ============================================================

W = np.array([
```

```
    [0.5, 0.8],
    [0.2, -0.4],
    [1.0, 0.3]
])

# ============================================================
# BIAS
# ============================================================

b = np.array([
    [0.2],
    [0.1],
    [-0.5]
])

# ============================================================
# LINEAR TRANSFORMATION
# ============================================================

z = W @ x + b

# ============================================================
# SIGMOID
# ============================================================

a = 1 / (1 + np.exp(-z))

# ============================================================
# RESULTS
# ============================================================

print("Input:")
print(x)

print("\nWeight matrix:")
print(W)

print("\nBias:")
print(b)

print("\nWeighted sums:")
print(z)

print("\nNeuron outputs:")
print(a)
```

```
PS C:\Users\Lenovo\ML Journey\01_binary_classification> py 14_hidden_layer.py
Input:
[[2.]
 [3.]]

Weight matrix:
[[ 0.5  0.8]
 [ 0.2 -0.4]
 [ 1.   0.3]]

Bias:
[[ 0.2]
 [ 0.1]
 [-0.5]]

Weighted sums:
[[ 3.6]
 [-0.7]
 [ 2.4]]

Neuron outputs:
[[0.97340301]
 [0.33181223]
 [0.9168273 ]]
```

Building the Complete Two-Layer Neural Network:

You've just built the **hidden layer**: $x \to W^{(1)}x + b^{(1)} \to \sigma \to a^{(1)}$
Now we'll add the **output layer**.
We'll use: $\boxed{2 \to 3 \to 1}$
meaning:

- **2 input features**
- **3 hidden neurons**
- **1 output neuron**

Forward propagation: (forward pass)
1. Hidden layer:
First:

$$\boxed{z^{(1)} = W^{(1)}x + b^{(1)}}$$

Then:

$$\boxed{a^{(1)} = \sigma\left(z^{(1)}\right)}$$

The hidden-layer output becomes the input to the output layer.
2. Output layer

$$\boxed{z^{(2)} = W^{(2)}a^{(1)} + b^{(2)}}$$

Then:
$15_forward_pass.py$So the complete forward pass is:

$$\boxed{x \to z^{(1)} \to a^{(1)} \to z^{(2)} \to \hat{y}}$$

This is something I want you to start thinking about naturally.
We have:

$$x \in \mathbb{R}^{2\times 1}$$

Three hidden neurons means:

$$W^{(1)} \in \mathbb{R}^{3\times 2}$$

and:

$$b^{(1)} \in \mathbb{R}^{3\times 1}$$

Therefore:

$$W^{(1)}x$$

has shape:

$$(3 \times 2)(2 \times 1) = 3 \times 1$$

So:

$$z^{(1)} \in \mathbb{R}^{3\times 1}$$

Then:

$$a^{(1)} \in \mathbb{R}^{3\times 1}$$

The output layer needs to transform 3 values into 1:

$$W^{(2)} \in \mathbb{R}^{1\times 3}$$

Therefore:

$$(1 \times 3)(3 \times 1) = 1 \times 1$$

This dimension reasoning becomes **critical** once we start building larger neural networks.

15_forward_pass.py

```
import numpy as np

# ============================================================
# 1. SIGMOID
# ============================================================

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# ============================================================
# 2. INPUT
# ============================================================

x = np.array([
    [2.0],
    [3.0]
])

# ============================================================
# 3. HIDDEN LAYER PARAMETERS
# ============================================================
```

```
W1 = np.array([
    [0.5, 0.8],
    [0.2, -0.4],
    [1.0, 0.3]
])

b1 = np.array([
    [0.2],
    [0.1],
    [-0.5]
])

# ============================================================
# 4. OUTPUT LAYER PARAMETERS
# ============================================================

W2 = np.array([
    [0.7, -0.5, 0.9]
])

b2 = np.array([
    [0.1]
])

# ============================================================
# 5. HIDDEN LAYER
# ============================================================

z1 = W1 @ x + b1

a1 = sigmoid(z1)

# ============================================================
# 6. OUTPUT LAYER
# ============================================================

z2 = W2 @ a1 + b2

y_hat = sigmoid(z2)

# ============================================================
# 7. PRINT EVERYTHING
# ============================================================

print("INPUT")
print("x =")
print(x)

print("\nHIDDEN LAYER")
print("W1 =")
print(W1)

print("\nb1 =")
print(b1)

print("\nz1 =")
print(z1)

print("\na1 =")
print(a1)

print("\nOUTPUT LAYER")
print("W2 =")
print(W2)

print("\nb2 =")
print(b2)

print("\nz2 =")
print(z2)

print("\ny_hat =")
print(y_hat)
```

```
PS C:\Users\Lenovo\ML Journey\01_binary_classification> py 15_forward_pass.py
INPUT
x =
[[2.]
 [3.]]

HIDDEN LAYER
W1 =
[[ 0.5  0.8]
 [ 0.2 -0.4]
 [ 1.   0.3]]

b1 =
[[ 0.2]
 [ 0.1]
 [-0.5]]

z1 =
[[ 3.6]
 [-0.7]
 [ 2.4]]

a1 =
[[0.97340301]
 [0.33181223]
 [0.9168273 ]]

OUTPUT LAYER
W2 =
[[ 0.7 -0.5  0.9]]

b2 =
[[0.1]]

z2 =
[[1.44062056]]

y_hat =
[[0.80855073]]
```

Our input is:

$$x = \begin{bmatrix} 2 \\ 3 \end{bmatrix}$$

The hidden weights are:

$$W^{(1)} = \begin{bmatrix} 0.5 & 0.8 \\ 0.2 & -0.4 \\ 1.0 & 0.3 \end{bmatrix}$$

and:

$$b^{(1)} = \begin{bmatrix} 0.2 \\ 0.1 \\ -0.5 \end{bmatrix}$$

Therefore:

$$z^{(1)} = W^{(1)}x + b^{(1)}$$

Hidden neuron 1

$$z_1^{(1)} = (0.5)(2) + (0.8)(3) + 0.2$$
$$= 1 + 2.4 + 0.2$$
$$\boxed{z_1^{(1)} = 3.6}$$

Therefore:

$$a_1^{(1)} = \sigma(3.6)$$
$$\boxed{a_1^{(1)} \approx 0.973}$$

Hidden neuron 2

$$z_2^{(1)} = (0.2)(2) + (-0.4)(3) + 0.1$$
$$= 0.4 - 1.2 + 0.1$$
$$\boxed{z_2^{(1)} = -0.7}$$

Therefore:

$$a_2^{(1)} = \sigma(-0.7)$$
$$\boxed{a_2^{(1)} \approx 0.332}$$

Hidden neuron 3

$$z_3^{(1)} = (1)(2) + (0.3)(3) - 0.5$$
$$= 2 + 0.9 - 0.5$$
$$\boxed{z_3^{(1)} = 2.4}$$

Therefore:

$$a_3^{(1)} = \sigma(2.4)$$
$$\boxed{a_3^{(1)} \approx 0.917}$$

So the hidden layer produces approximately:

$$a^{(1)} = \begin{bmatrix} 0.973 \\ 0.332 \\ 0.917 \end{bmatrix}$$

Output neuron
Our output weights are:

$$W^{(2)} = [0.7 \quad -0.5 \quad 0.9]$$

Therefore:

$$z^{(2)} = 0.7(0.973) - 0.5(0.332) + 0.9(0.917) + 0.1$$

Approximately:

$$z^{(2)} \approx 1.40$$

Then:

$$\hat{y} = \sigma(1.40)$$

so:

$$\boxed{\hat{y} \approx 0.80}$$

Given this input, the estimated probability of class 1 is about 80%.

Backpropagation:
Suppose: $W^{(1)}$ changes slightly.
That changes: $z^{(1)}$
which changes: $a^{(1)}$
which changes: $z^{(2)}$
which changes: $\hat{y}$
which changes: L
Therefore: $\frac{\partial L}{\partial W^{(1)}}$
requires a chain of derivatives: $\boxed{\frac{\partial L}{\partial W^{(1)}} = \frac{\partial L}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z^{(2)}} \frac{\partial z^{(2)}}{\partial a^{(1)}} \frac{\partial a^{(1)}}{\partial z^{(1)}} \frac{\partial z^{(1)}}{\partial W^{(1)}}}$
That chain rule is backpropagation.

Our network is: $2 \to 3 \to 1$
We have:

$$z_1 = W_1 X + b_1$$
$$a_1 = \sigma(z_1)$$
$$z_2 = W_2 a_1 + b_2$$
$$\hat{y} = \sigma(z_2)$$
$$L = -[y \log \hat{y} + (1 - y) \log(1 - \hat{y})]$$

1. Output-layer gradient
We've already derived this for logistic regression:

$$\boxed{\frac{\partial L}{\partial z_2} = \hat{y} - y}$$

Define: $\boxed{\delta_2 = \hat{y} - y}$
Then: $\boxed{\frac{\partial L}{\partial W_2} = \delta_2 a_1^T}$ and $\boxed{\frac{\partial L}{\partial b_2} = \delta_2}$

2. Backpropagate into the hidden layer

The output error must travel backward through W_2: $\frac{\partial L}{\partial a_1} = W_2^T \delta_2$

But a_1 came from: $a_1 = \sigma(z_1)$

and: $\sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big)$

Therefore: $\boxed{\delta_1 = (W_2^T \delta_2) \odot a_1(1 - a_1)}$

where $\odot$ means element-wise multiplication.

Then: $\boxed{\frac{\partial L}{\partial W_1} = \delta_1 X^T}$ and $\boxed{\frac{\partial L}{\partial b_1} = \delta_1}$

16_neural_network_xor.py

```
import numpy as np
import matplotlib.pyplot as plt

# ============================================================
# 1. XOR DATASET
# ============================================================

X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
], dtype=float)

y = np.array([
    [0],
    [1],
    [1],
    [0]
], dtype=float)

# ============================================================
# 2. ACTIVATION FUNCTIONS
# ============================================================

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def binary_cross_entropy(y, p):

    epsilon = 1e-15

    p = np.clip(p, epsilon, 1 - epsilon)

    return -np.mean(
        y * np.log(p) +
        (1 - y) * np.log(1 - p)
    )

# ============================================================
# 3. INITIALIZE PARAMETERS
# ============================================================

np.random.seed(42)

# 2 inputs → 3 hidden neurons
W1 = np.random.randn(2, 3) * 0.5

b1 = np.zeros((1, 3))

# 3 hidden neurons → 1 output
W2 = np.random.randn(3, 1) * 0.5

b2 = np.zeros((1, 1))

# ============================================================
# 4. TRAINING SETTINGS
# ============================================================

learning_rate = 1.0

epochs = 10000

loss_history = []
```

```
# ============================================================
# 5. TRAINING LOOP
# ============================================================

for epoch in range(epochs):

    # ------------------------------------------------------
    # FORWARD PASS
    # ------------------------------------------------------

    z1 = X @ W1 + b1

    a1 = sigmoid(z1)

    z2 = a1 @ W2 + b2

    y_hat = sigmoid(z2)

    # ------------------------------------------------------
    # LOSS
    # ------------------------------------------------------

    loss = binary_cross_entropy(y, y_hat)

    loss_history.append(loss)

    # ------------------------------------------------------
    # BACKPROPAGATION
    # ------------------------------------------------------

    # Output layer
    dz2 = y_hat - y

    dW2 = (a1.T @ dz2) / len(X)

    db2 = np.mean(dz2, axis=0, keepdims=True)

    # Hidden layer
    dz1 = (
        (dz2 @ W2.T)
        * a1
        * (1 - a1)
    )

    dW1 = (X.T @ dz1) / len(X)

    db1 = np.mean(dz1, axis=0, keepdims=True)

    # ------------------------------------------------------
    # PARAMETER UPDATE
    # ------------------------------------------------------

    W2 -= learning_rate * dW2

    b2 -= learning_rate * db2

    W1 -= learning_rate * dW1

    b1 -= learning_rate * db1

# ============================================================
# 6. FINAL PREDICTIONS
# ============================================================

z1 = X @ W1 + b1

a1 = sigmoid(z1)

z2 = a1 @ W2 + b2

y_hat = sigmoid(z2)

y_pred = (y_hat >= 0.5).astype(int)

# ============================================================
```

```
# 7. RESULTS
# ============================================================

print("\nFINAL RESULTS")
print("=" * 50)

for i in range(len(X)):

    print(
        f"Input: {X[i]} | "
        f"Probability: {y_hat[i,0]:.4f} | "
        f"Prediction: {y_pred[i,0]} | "
        f"Actual: {int(y[i,0])}"
    )

accuracy = np.mean(y_pred == y)

print("\nAccuracy:", accuracy)

# ============================================================
# 8. LOSS CURVE
# ============================================================

plt.figure(figsize=(8, 5))

plt.plot(loss_history)

plt.xlabel("Epoch")

plt.ylabel("Binary Cross-Entropy Loss")

plt.title("Neural Network Learning on XOR")

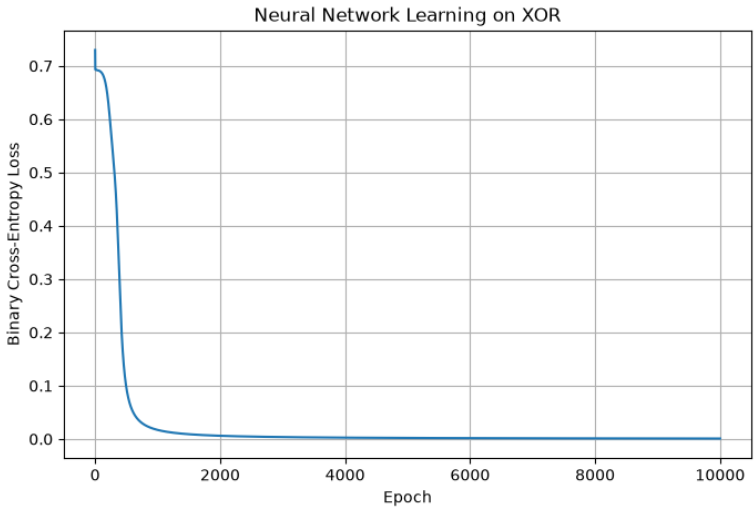
plt.grid(True)

plt.show()
```

```
PS C:\Users\Lenovo\ML Journey\01_binary_classification> py 16_neural_network_xor.py

FINAL RESULTS
==================================================
Input: [0. 0.] | Probability: 0.0011 | Prediction: 0 | Actual: 0
Input: [0. 1.] | Probability: 0.9991 | Prediction: 1 | Actual: 1
Input: [1. 0.] | Probability: 0.9991 | Prediction: 1 | Actual: 1
Input: [1. 1.] | Probability: 0.0009 | Prediction: 0 | Actual: 0

Accuracy: 1.0
```



A hidden neuron creates a nonlinear transformation:

$$a_1 = \sigma(w_1^T x + b_1)$$
$$a_2 = \sigma(w_2^T x + b_2)$$
$$a_3 = \sigma(w_3^T x + b_3)$$

The output combines these learned features:

$$\hat{y} = \sigma(v_1 a_1 + v_2 a_2 + v_3 a_3 + b)$$

So the network is effectively learning: simple nonlinear features → combine features → complex decision

17_xor_nonlinear_decision_boundary

```
import numpy as np
import matplotlib.pyplot as plt

# ============================================================
# 1. DATA: XOR
# ============================================================

X = np.array([
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
], dtype=float)

y = np.array([
    [0],
    [1],
    [1],
    [0]
], dtype=float)

# ============================================================
# 2. ACTIVATIONS AND LOSS
# ============================================================

def relu(z):
    return np.maximum(0, z)

def relu_derivative(z):
    return (z > 0).astype(float)

def sigmoid(z):
    # Clip to avoid overflow in exp for extreme values.
    z = np.clip(z, -500, 500)
    return 1.0 / (1.0 + np.exp(-z))

def binary_cross_entropy(y_true, probabilities):
    eps = 1e-15
    probabilities = np.clip(probabilities, eps, 1 - eps)
    return -np.mean(
        y_true * np.log(probabilities)
        + (1 - y_true) * np.log(1 - probabilities)
    )

# ============================================================
# 3. INITIALIZATION
# ============================================================

np.random.seed(42)

input_size = 2
hidden1_size = 8
hidden2_size = 8
output_size = 1

W1 = np.random.randn(input_size, hidden1_size) * np.sqrt(2 / input_size)
b1 = np.zeros((1, hidden1_size))

W2 = np.random.randn(hidden1_size, hidden2_size) * np.sqrt(2 / hidden1_size)
b2 = np.zeros((1, hidden2_size))

W3 = np.random.randn(hidden2_size, output_size) * 0.1
b3 = np.zeros((1, output_size))

# ============================================================
# 4. FORWARD PASS
# ============================================================

def forward(X_input):
    z1 = X_input @ W1 + b1
```

```
    a1 = relu(z1)

    z2 = a1 @ W2 + b2
    a2 = relu(z2)

    z3 = a2 @ W3 + b3
    probabilities = sigmoid(z3)

    # Return intermediate values for backpropagation.
    cache = (z1, a1, z2, a2, z3, probabilities)
    return probabilities, cache

# ============================================================
# 5. TRAINING
# ============================================================

learning_rate = 0.1
epochs = 5000
lambda_reg = 0.001
m = len(X)

loss_history = []

for epoch in range(epochs):
    # Forward pass
    y_hat, cache = forward(X)
    z1, a1, z2, a2, z3, y_hat = cache

    # Loss = data loss + L2 weight penalty
    data_loss = binary_cross_entropy(y, y_hat)
    reg_loss = (lambda_reg / 2) * (
        np.sum(W1 ** 2) + np.sum(W2 ** 2) + np.sum(W3 ** 2)
    )
    loss_history.append(data_loss + reg_loss)

    # Backpropagation: sigmoid + binary cross-entropy
    dz3 = y_hat - y
    dW3 = (a2.T @ dz3) / m + lambda_reg * W3
    db3 = np.mean(dz3, axis=0, keepdims=True)

    dz2 = (dz3 @ W3.T) * relu_derivative(z2)
    dW2 = (a1.T @ dz2) / m + lambda_reg * W2
    db2 = np.mean(dz2, axis=0, keepdims=True)

    dz1 = (dz2 @ W2.T) * relu_derivative(z1)
    dW1 = (X.T @ dz1) / m + lambda_reg * W1
    db1 = np.mean(dz1, axis=0, keepdims=True)

    # Gradient descent
    W3 -= learning_rate * dW3
    b3 -= learning_rate * db3

    W2 -= learning_rate * dW2
    b2 -= learning_rate * db2

    W1 -= learning_rate * dW1
    b1 -= learning_rate * db1

# ============================================================
# 6. FINAL PREDICTIONS
# ============================================================

y_hat, _ = forward(X)
y_pred = (y_hat >= 0.5).astype(int)

print("\nFINAL XOR RESULTS")
print("=" * 65)
for i in range(len(X)):
    print(
        f"Input = {X[i].astype(int)} | "
        f"Probability = {y_hat[i, 0]:.4f} | "
        f"Prediction = {y_pred[i, 0]} | "
        f"Actual = {int(y[i, 0])}"
    )

accuracy = np.mean(y_pred == y)
```

```
print(f"\nAccuracy: {accuracy:.2%}")
print(f"Final loss: {loss_history[-1]:.6f}")

# ============================================================
# 7. CREATE A MESH GRID FOR THE DECISION BOUNDARY
# ============================================================

# Expand beyond [0, 1] so the shape of the boundary is visible.
x1_min, x1_max = -0.5, 1.5
x2_min, x2_max = -0.5, 1.5

grid_step = 0.01
xx1, xx2 = np.meshgrid(
    np.arange(x1_min, x1_max + grid_step, grid_step),
    np.arange(x2_min, x2_max + grid_step, grid_step)
)

# Each grid location is one candidate input [x1, x2].
grid_points = np.column_stack((xx1.ravel(), xx2.ravel()))

# Evaluate the trained network at every grid location.
grid_probabilities, _ = forward(grid_points)
Z = grid_probabilities.reshape(xx1.shape)

# ============================================================
# 8. PLOT THE NONLINEAR DECISION BOUNDARY
# ============================================================

plt.figure(figsize=(8, 7))

# Show the model's predicted probability over the plane.
contour = plt.contourf(
    xx1, xx2, Z,
    levels=np.linspace(0, 1, 21),
    cmap="coolwarm",
    alpha=0.75
)
plt.colorbar(contour, label="Predicted probability P(y = 1)")

# The 0.5 contour is the classification decision boundary.
boundary = plt.contour(
    xx1, xx2, Z,
    levels=[0.5],
    colors="black",
    linewidths=2.5
)
plt.clabel(boundary, fmt={0.5: "Boundary: p = 0.5"}, inline=True, fontsize=9)

# Plot the four actual XOR samples.
plt.scatter(
    X[:, 0], X[:, 1],
    c=y.ravel(),
    cmap="coolwarm",
    vmin=0, vmax=1,
    edgecolors="black",
    s=180,
    zorder=3
)

for i, (x1, x2) in enumerate(X):
    plt.annotate(
        f"y={int(y[i, 0])}",
        (x1, x2),
        xytext=(7, 7),
        textcoords="offset points",
        fontsize=10,
        weight="bold"
    )

plt.xlim(x1_min, x1_max)
plt.ylim(x2_min, x2_max)
plt.xlabel("Input x1")
plt.ylabel("Input x2")
plt.title("MLP Learned Nonlinear Decision Boundary for XOR")
plt.grid(alpha=0.2)
plt.tight_layout()
```

```
plt.show()

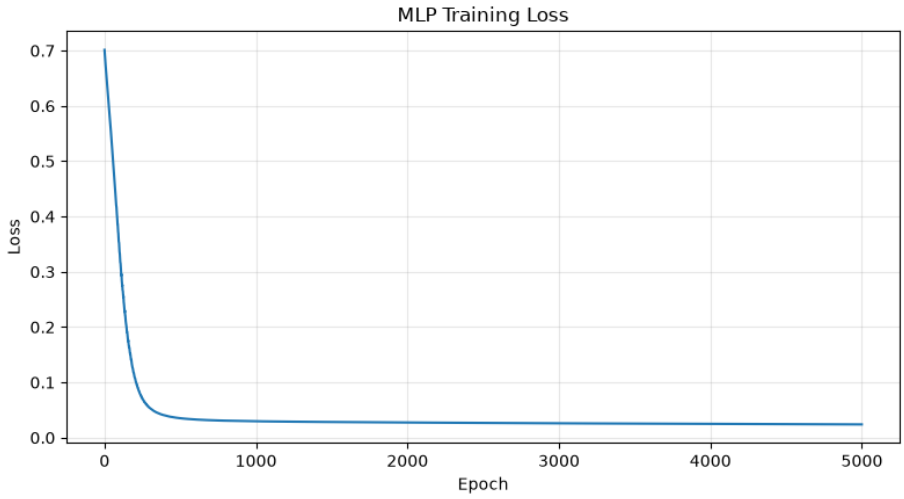
# ============================================================
# 9. PLOT TRAINING LOSS
# ============================================================

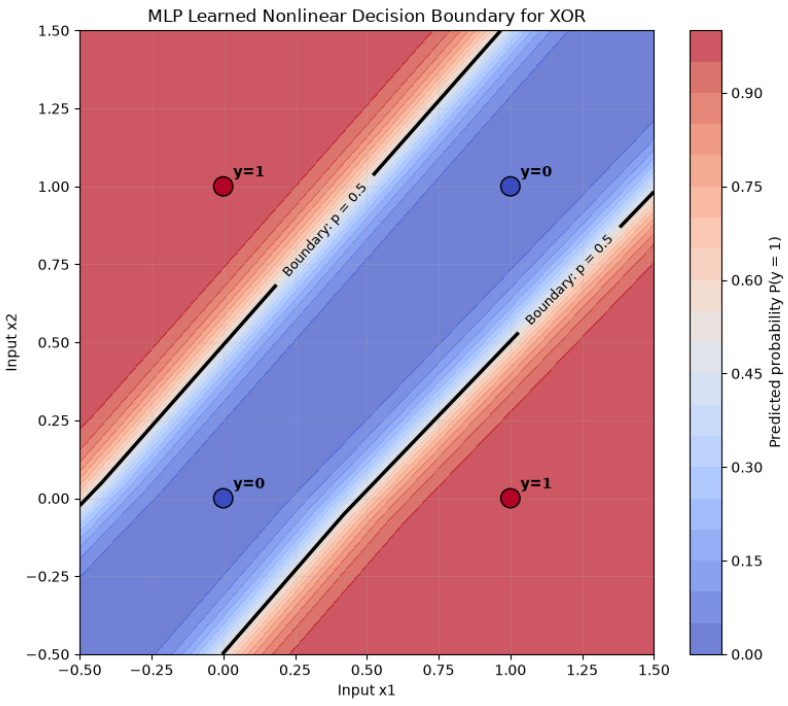
plt.figure(figsize=(8, 4.5))
plt.plot(loss_history)
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("MLP Training Loss")
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

```
PS C:\Users\Lenovo\ML Journey\01_binary_classification> py 17_xor_nonlinear_decision_boundary.py

FINAL XOR RESULTS
================================================================
Input = [0 0] | Probability = 0.0028 | Prediction = 0 | Actual = 0
Input = [0 1] | Probability = 0.9977 | Prediction = 1 | Actual = 1
Input = [1 0] | Probability = 0.9985 | Prediction = 1 | Actual = 1
Input = [1 1] | Probability = 0.0017 | Prediction = 0 | Actual = 0

Accuracy: 100.00%
Final loss: 0.023941
```





Input data
$X \in \mathbb{R}^{4\times 2}$
Four examples, two input bits each

⬇

Hidden layer 1: 8 neurons
$A_1 = \text{ReLU}(XW_1 + b_1)$

⬇

Hidden layer 2: 8 neurons
$A_2 = \text{ReLU}(A_1W_2 + b_2)$

⬇

Output layer: 1 neuron
$\hat{y} = \sigma(A_2W_3 + b_3)$

⬇

Prediction
$Probability\ atleast\ 0.5 \rightarrow class1; otherwise \rightarrow class0.$